Cross-Domain Evaluation of Diamond-DETR for Surface-Defect Detection

Transformers versus CNNs on the Magnetic Tile Defect Dataset

Deep Vision, Comparison of Transformers and CNNs

Adarsh Koshiya

a.koshiya@oth-aw.de

Ostbayerische Technische Hochschule (OTH) Amberg-Weiden

Deep Vision · Sommersemester 2026

Abstract. We ask whether the architectural choices behind *Diamond-DETR*, a transformer detector designed for synthetic-diamond surface inspection, transfer to a different industrial domain: magnetic tile surface defects. Four detectors are trained and evaluated under matched conditions on a five-class defect task (blowhole, break, crack, fray, uneven) with severe class imbalance. Under transfer learning the transformer RT-DETR-r18 reaches a test mAP@0.5 of 0.763, nearly double the best CNN (YOLOv8n, 0.403), and, more importantly, spreads accuracy evenly across all five classes (per-class mAP@[0.5:0.95] from 0.38 to 0.58) where both CNNs collapse on the rare *break* and *fray* defects (AP $\approx$ 0.02 to 0.08). The transformer recall of 0.82 against the CNNs' 0.23 to 0.35 is the decisive property for inspection, where a missed defect ships a faulty part. The cost is speed (40 against 150 to 205 FPS) and size (19.9 M against 2.6 to 3.0 M parameters). On the central architectural question we report the *initialisation-controlled* result: against a from-scratch RT-DETR-r18 trained under the identical recipe (mAP@0.5 0.763, mAP@[0.5:0.95] 0.514), Diamond-DETR is tied on mAP@0.5 (0.762) and marginally behind on the stricter mAP@[0.5:0.95] (0.489 against 0.514) while using 28% fewer parameters (14.3 against 19.9 M). Diamond-DETR's contribution therefore reproduces cross-domain as *efficiency at near-parity accuracy*, not a strict accuracy win; the higher 0.489 against 0.446 reading compares it against a diamond-pretrained RT-DETR and is confounded by initialisation. A complete component ablation, reproducing the paper's full Table 2 design, exposes the limit of transfer: the HS-FPN neck that is the paper's strongest component on diamonds is detrimental on tiles, so the best tile configuration (A+C+D, 0.587) surpasses the full Diamond-DETR (0.489).

June 2026 · All reported numbers trace to `results/metrics/*.json`.

Trained weights (all four models): https://git.oth-aw.de/40a2/deep-vision

Contents

1 Introduction and research question **2**

2 Background **2**

3 Dataset and preparation **2**

4 Models compared and provenance **3**

4.1 How each model works, what trains it, and where it comes from 4

5 Experimental setup **5**

6 Results **5**

6.1 Initialisation-controlled comparison (Diamond-DETR against RT-DETR) 7

6.2 Component ablation: the complete factorial 8

7 Discussion **10**

7.1 Transformers against CNNs 10

7.2 Diamond-DETR against RT-DETR 10

8 Grad-CAM attention analysis **11**

9 Cross-domain comparison **11**

10 Limitations **12**

11 Future work **12**

12 Conclusion **13**

A Data integrity and engineering notes **13**

B Fork compatibility patches **14**

References **14**

1 Introduction and research question

Surface-defect detection is a quality-critical task in industrial inspection: the dominant cost is a *missed* defect, since a false alarm only triggers a re-check while an undetected flaw ships a faulty part. This asymmetry makes recall, and balanced performance across rare defect types, matter more than a single aggregate accuracy number.

Diamond-DETR [1] is a transformer detector published for synthetic-diamond surface inspection. It modifies the real-time detection transformer RT-DETR [2] with three lightweight components and claims to be "lightweight yet at least as strong" as its baseline. This study tests whether that claim, and more generally the transformer advantage, transfers to a different industrial domain: the Magnetic Tile Surface Defect dataset [7].

We compare two transformer detectors (Diamond-DETR and RT-DETR-r18) against two compact CNN detectors (YOLOv8n and YOLO11n) under matched training conditions, and ask:

1. Do transformer detectors outperform compact CNN detectors on small, imbalanced industrial defect data, and where does any advantage come from?
2. Does Diamond-DETR's advantage over its own RT-DETR baseline survive the domain change, and which of its three component swaps actually earns its place?

The analysis follows a single reproducible pipeline: we frame the problem as five-class detection, convert segmentation masks to boxes, train under one harness, and read all final metrics from machine-written JSON artifacts so that no number in this document is entered by hand.

2 Background

RT-DETR [2] is a real-time detection transformer: a ResNet backbone feeds an efficient hybrid encoder (the *AIFI* attention module on the deepest feature map plus a CNN-based cross-scale fusion), and a transformer decoder performs *set prediction*. Following DETR [3], the decoder emits one query per object, is matched to ground truth by a bipartite (Hungarian) assignment, and requires *no* non-maximum suppression. This global, NMS-free formulation is the structural difference from anchor-based CNN heads.

Diamond-DETR [1] keeps RT-DETR's AIFI encoder and decoder head and swaps three components, summarised in Table 2: a re-parameterisable FasterNet backbone block [4, 5], a high-level screening feature-pyramid neck with channel attention [6], and a re-parameterisable fusion block. The intent is higher accuracy at fewer parameters.

YOLOv8n and YOLO11n [8] are compact anchor-free single-stage CNN detectors (C2f and C3k2 backbones respectively, PAN-style necks, decoupled detection heads). They are the established and current real-time baselines.

3 Dataset and preparation

The Magnetic Tile Surface Defect dataset [7] contains 1,344 grayscale tile images, collected from a production line and annotated with pixel-level saliency masks, with five defect types plus a large defect-free class. We frame the task as five-class detection (literature precedent exists), with the 952 defect-free tiles acting as negatives with empty label files. Table 1 shows the class distribution; the imbalance is severe, with *fray* represented by only 32 images.

Table 1. Class distribution of the Magnetic Tile dataset. The imbalance, especially the 32 fray images, motivates a per-class analysis rather than a single aggregate metric.

	blowhole	**break**	**crack**	**fray**	**uneven**	**defect-free**
Images	115	85	57	32	103	952
Box instances	118	117	70	38	105	0 (negatives)

Masks to boxes (the data philosophy). The dataset ships pixel segmentation masks, not boxes. We convert each mask to YOLO-format boxes by connected-component analysis: one box per connected defect region with area at least 20 px, the box taken as the component's bounding rectangle. A whole-image fallback box is emitted only when a mask yields no region, which fired on just 6 of 392 defect images. This step is deliberately lossy in one way: two touching defect regions can merge into a single box, which is recorded as a limitation (Section 10).

Class imbalance. Imbalance is the central, and industrially realistic, difficulty of this dataset: real defects are rare and unevenly distributed. We handle it through a stratified split that keeps the rare classes representative across folds, by retaining the 952 clean tiles as explicit negatives (which suppresses false positives), and by reporting *per-class* AP as the primary diagnostic rather than hiding imbalance behind a single mAP. We deliberately apply no loss re-weighting or over-sampling, so that the comparison reflects the architectures under a common objective; targeted rebalancing is a promising follow-up (Section 11).

Data augmentation. Augmentation is the main defence against overfitting on 938 images, and the exact recipe is recorded in each run's `args.yaml`. Both families share a conservative geometric and photometric core: HSV jitter (h=0.015, s=0.7, v=0.4), translation ($\pm$0.1), scaling ($\pm$0.5), and horizontal flips (p=0.5), with rotation, shear, perspective, and vertical flips disabled on purpose because a magnetic tile has a consistent orientation under the camera. The CNN family additionally runs the heavier Ultralytics stack (mosaic, closed for the final 10 epochs; RandAugment; and random erasing at p=0.4), which the forked RT-DETR pipeline does not. The transformers therefore see *less* augmentation diversity than the CNNs, which makes the transformer result conservative rather than inflated; the asymmetry is recorded as a confound in Section 10.

Split. A stratified 70/20/10 split with seed 42 produces 938 training, 268 validation, and 138 test images, with zero image overlap between splits (verified). All accuracy numbers in this report are measured on the held-out test split.

4 Models compared and provenance

Table 2. The four detectors. Diamond-DETR is RT-DETR-r18 with three components swapped (B, C, D); it shares the AIFI encoder and the set-prediction decoder head with RT-DETR.

Model	**Family**	**Params**	**Role in the study**
Diamond-DETR	Transformer	14.3 M	The paper model under test
RT-DETR-r18	Transformer	19.9 M	The baseline Diamond-DETR improves upon (isolates its contribution)
YOLOv8n	CNN	3.0 M	Established real-time CNN detector
YOLO11n	CNN	2.6 M	Current-generation CNN detector

Diamond-DETR's three swaps, relative to RT-DETR-r18, are:

- **(B)** `BasicBlock_Faster_Block_Rep` backbone blocks (re-parameterisable FasterNet convolutions) replacing the standard ResNet-18 `BasicBlock`, at the same channel dimensions (64/128/256/512);
- **(C)** `ChannelAttention_HSFPN`, a channel-attention high-level screening feature pyramid [6], replacing the standard feature-pyramid neck;
- **(D)** `DRBC3`, a re-parameterisable fusion block, replacing the `RepC3` fusion blocks.

What is reused versus built here. The four model architectures, the forked Ultralytics 8.0.201 that defines the DETR custom modules, RT-DETR's diamond-pretrained checkpoint, and the COCO-pretrained YOLO weights are reused from the upstream repositories [1, 8]. Built for this study are the data-preparation script (mask to box plus stratified split), the training harness, the metrics writer, the component-ablation configurations and runner, six compatibility patches that let the unmaintained fork run on modern PyTorch and NumPy, and the Grad-CAM analysis. The trained checkpoints and every test-split number in Section 6 were produced and measured here.

4.1 How each model works, what trains it, and where it comes from

Table 3 is the provenance map: for each detector it states the mechanism, the script and recipe that trained it, and the weights it started from. The two transformers share the AIFI attention encoder and the `RTDETRDecoder` set-prediction head (one query per object, Hungarian matching, no NMS); they differ only in backbone and neck. All four are evaluated on the test split by `evaluate.py` (`--type rtdetr` for the DETR models, which require the fork on `PYTHONPATH`; `--type yolo` for the CNNs).

Table 3. Per-model mechanism, training recipe, and initialisation. "train.sh, fork" is the forked Ultralytics 8.0.201 run from `code/diamond_detr/RTDETR-main`; "stock" is the pip-installed Ultralytics 8.4.x.

Model	How it works (mechanism)	Trained by	Initialised from
Diamond-DETR	RT-DETR-r18 with three swapped components: Rep-FasterNet backbone (B), HS-FPN channel-attention neck (C), `DRBC3` fusion (D); shared AIFI encoder and decoder.	train.sh, fork; AdamW, 200 ep, batch 4	Random init (authors published no weights)
RT-DETR-r18	ResNet-18 backbone (`ConvNormLayer` + `BasicBlock`), standard FPN + `RepC3` neck, AIFI encoder, `RTDETRDecoder` head.	train.sh, fork; AdamW, 200 ep, batch 4	Paper's `best.pt` (diamond-pretrained, mAP@0.5 0.89); partial load via `intersect_dicts`, head re-init for nc=5
YOLOv8n	Anchor-free single-stage CNN: C2f backbone, PAN-FPN neck, decoupled head with DFL.	train.sh, stock; SGD, 150 ep, batch 4	`yolov8n.pt` (COCO-pretrained)
YOLO11n	Newer Ultralytics CNN: C3k2 backbone, C2PSA attention block, decoupled head.	train.sh, stock; SGD, 150 ep, batch 4	`yolo11n.pt` (COCO-pretrained)

5 Experimental setup

Both families share the image size (640×640), batch size (4), seed (42), and a learning rate of 10^{-4}. They differ in the two variables that model-family practice dictates, summarised in Table 4. In our setup the transformers needed AdamW: an initial SGD run left both at approximately zero mAP. We read this as specific to training Diamond-DETR from random initialisation on a small, imbalanced five-class set, not as a universal rule. Notably the source paper trained the same architectures with SGD (Table 1 of [1], lr 10^{-4}, momentum 0.8) and reached 88 to 90% mAP@0.5 on its binary diamond task, so the optimiser sensitivity we observed is regime-specific. The pretrained CNNs converge by 150 epochs; the transformers were given 200.

Table 4. Training configuration. The optimiser and epoch-count differences are deliberate and follow model-family practice; they are discussed as a confound in Section 10.

Setting	**CNNs (YOLOv8n, YOLO11n)**	**Transformers (RT-DETR, Diamond-DETR)**
Optimiser	SGD ($\mu = 0.8$)	AdamW ($\mu = 0.9$)
Epochs	150	200
Batch / image	4 / 640	4 / 640
Learning rate	10^{-4}	10^{-4}
Initialisation	COCO-pretrained	RT-DETR: diamond-pretrained; Diamond-DETR: random

Two-environment architecture. The DETR family runs on a forked Ultralytics 8.0.201 (which defines the custom modules), and the YOLO family runs on stock Ultralytics 8.4.x. The two versions cannot share one Python process, so the training harness runs each model in a separate subprocess and writes test metrics to JSON; all downstream analysis only reads those JSON files. Six compatibility patches were applied to the fork (for example `np.trapz` to `np.trapezoid`, and a `torch.load` default change), all of which are compatibility fixes, not model changes.

Initialisation note. RT-DETR-r18 was initialised from the paper's own checkpoint, trained on a two-class diamond task; its backbone and neck transfer through a partial load while the five-class head is re-initialised. Diamond-DETR has no public pretrained weights anywhere in the repository, so it trains from *random initialisation*. It converged regardless (validation mAP@0.5 climbed from ≈ 0.00001 to 0.80 over 200 epochs). Because this asymmetry favours RT-DETR, we additionally trained a from-scratch RT-DETR-r18 under the identical recipe (Section 6.1) to obtain a fair, initialisation-controlled architectural comparison.

6 Results

Table 5 reports the headline test-split metrics and Figure 1 plots the two aggregate accuracies. Table 6 and Figure 2 give the per-class breakdown, which is the core finding of the study. Figure 3 places the four models on the accuracy versus speed plane.

Table 5. Test-split results (138 images, seed 42). Best value per column in bold. Transformers use AdamW for 200 epochs; CNNs use SGD for 150 epochs.

Model	mAP@0.5	mAP@[.5:.95]	Precision	Recall	FPS	Params
Diamond-DETR (random init)	0.762	0.489	0.745	0.746	56.5	**14.3 M**
RT-DETR-r18 (pretrained)	0.763	0.446	0.733	0.819	40.2	19.9 M
YOLOv8n (COCO)	0.403	0.275	0.613	0.349	204.7	3.0 M
YOLO11n (COCO)	0.335	0.204	0.579	0.227	153.2	2.6 M

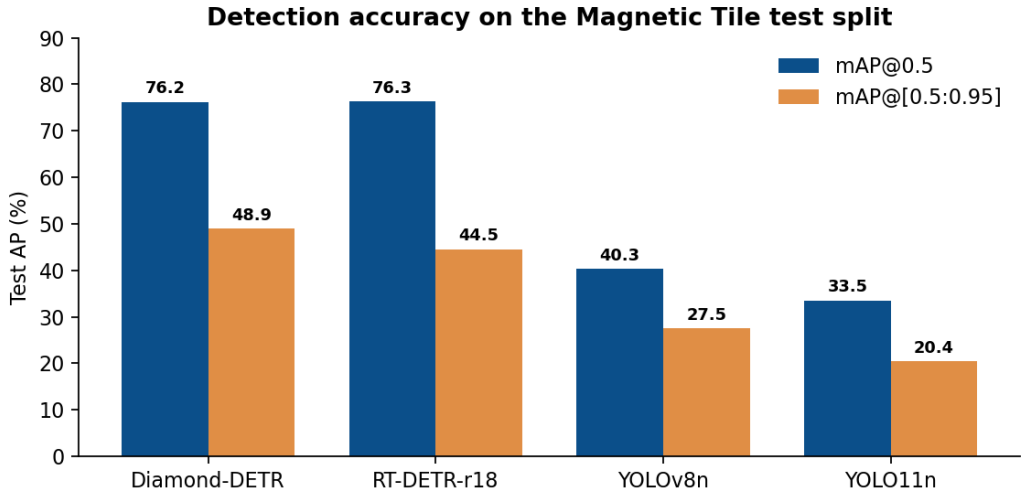


Figure 1. Aggregate detection accuracy. Both transformers reach roughly 0.76 mAP@0.5; the CNNs sit near 0.34 to 0.40.

Table 6. Per-class AP@[0.5:0.95] on the test split. The CNNs are competitive only on the large, high-contrast *blowhole* class and nearly collapse on the rare, elongated classes; both transformers keep every class at or above 0.36.

Model	blowhole	break	crack	fray	uneven
Diamond-DETR	0.670	0.502	0.536	0.383	0.356
RT-DETR-r18	0.578	0.415	0.388	0.463	0.383
YOLOv8n	0.707	0.074	0.280	0.069	0.244
YOLO11n	0.563	0.078	0.175	0.023	0.181

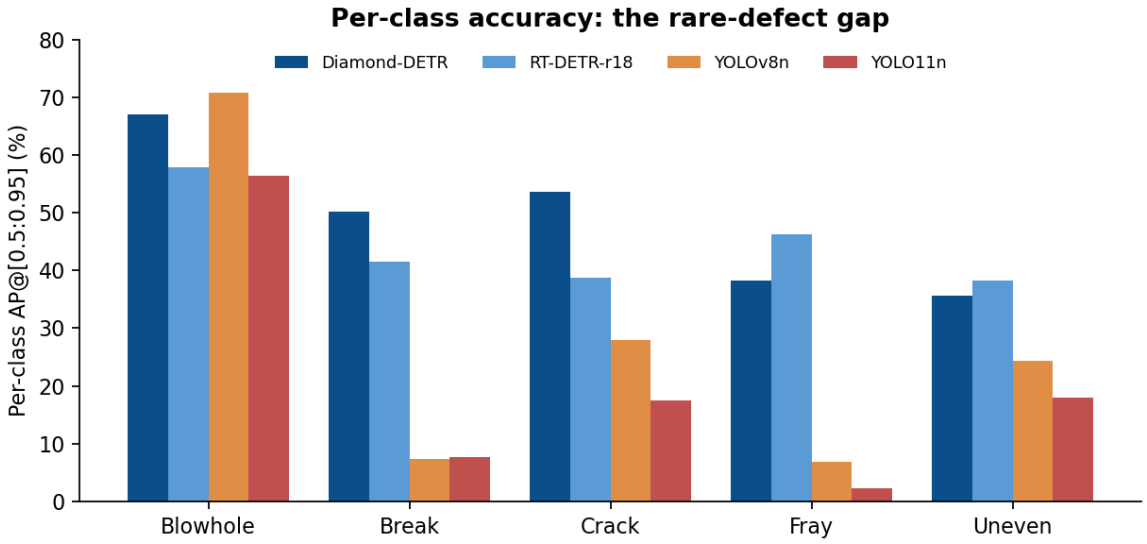


Figure 2. Per-class accuracy. On the rarest class *fray* (32 training images) RT-DETR scores 0.463 against YOLOv8n's 0.069, a factor of about 6.7, and against YOLO11n's 0.023, a factor of about 20. This is the headline of the study: the transformers convert class imbalance from a near-total failure into a manageable spread.

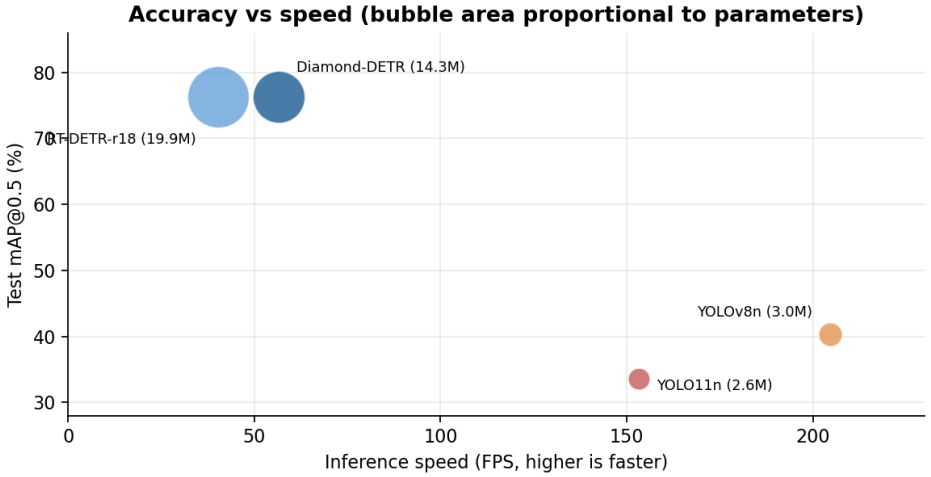


Figure 3. Accuracy against speed; bubble area is proportional to parameter count. The transformers buy accuracy and balanced recall at roughly five times the latency and seven times the parameters of the CNNs, yet remain real-time at 40 to 56 FPS.

Validation against test is consistent: RT-DETR reached validation mAP@0.5 0.731 at epoch 200 and test 0.763; Diamond-DETR reached validation 0.797 and test 0.762. In wall-clock terms the per-image latencies are roughly 18 ms (Diamond-DETR), 25 ms (RT-DETR), 5 ms (YOLOv8n), and 7 ms (YOLO11n), so every model clears real-time for line-scan inspection.

6.1 Initialisation-controlled comparison (Diamond-DETR against RT-DETR)

The headline table compares a from-scratch Diamond-DETR against a diamond-pretrained RT-DETR, which flatters Diamond-DETR. With a from-scratch RT-DETR-r18 baseline trained under the identical recipe we can compare like with like (Table 7, Figure 4). Under matched initialisation the two transformers are tied on mAP@0.5, RT-DETR is slightly ahead on mAP@[0.5:0.95] (0.514

against 0.489), and Diamond-DETR reaches near-parity at 28% fewer parameters. Diamond-DETR's genuine contribution on this domain is therefore efficiency at essentially equal detection quality, a favourable accuracy and size trade, rather than the outright accuracy win the headline table appears to show. A second observation falls out of the same baseline: the from-scratch RT-DETR (mAP@[0.5:0.95] 0.514) outscores the diamond-pretrained RT-DETR (0.446) on box tightness, suggesting that on this dataset diamond-domain pretraining mainly bought recall (0.819) at some cost to localisation.

Table 7. Initialisation-controlled comparison, both transformers from random init under the identical recipe.

Both from random init (AdamW, 200 ep)	**mAP@0.5**	**mAP@[.5:.95]**	**Params**
RT-DETR-r18 (from scratch)	0.763	0.514	19.9 M
Diamond-DETR (from scratch, full)	0.762	0.489	**14.3 M**

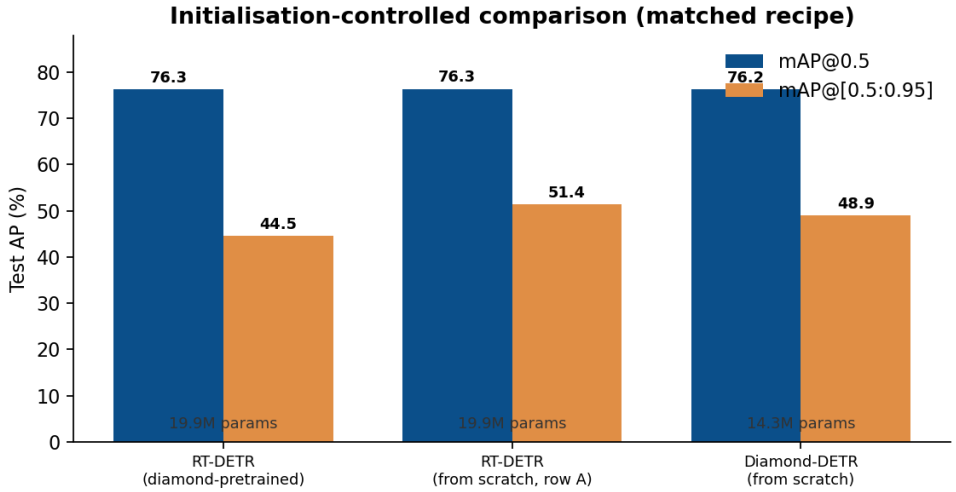


Figure 4. Under matched initialisation the two transformers are accuracy-tied; Diamond-DETR's advantage is parameter efficiency (14.3 against 19.9 M), not a strict accuracy gain. The 0.489 against 0.446 reading in Table 5 compares against a pretrained RT-DETR and is confounded by initialisation.

6.2 Component ablation: the complete factorial

To attribute Diamond-DETR's behaviour to its three component swaps (backbone B, neck C, fusion D), we trained all eight combinations from random init under the identical recipe. This is a complete 2^3 factorial: the same eight-configuration design the source paper reports in its Table 2 [1], which lets us compare each component's contribution across the two domains directly. The configurations and our test-split metrics are in Table 8.

The headline is positive and simple: the best configuration we found (A+C+D at 0.587 mAP@[0.5:0.95], essentially tied with A+B at 0.577) beats both the full Diamond-DETR (0.489) and the from-scratch RT-DETR baseline (0.514 in the initialisation-controlled comparison above). An ablation-guided trim of Diamond-DETR therefore outperforms the published full model on this domain.

Reading across the eight rows, the fusion block `DRBC3` (D) is the only swap that helps in most contexts (three of its four pairings), the HS-FPN neck (C) hurts in most contexts (three of four), and the backbone (B) gives the single largest jump from the bare baseline (+0.063) but turns detrimental in combination. The full model is beaten by smaller subsets (both A+C+D at 0.587 and A+B+D at 0.541 exceed its 0.489), so its one genuine advantage is the smallest

Table 8. Complete 2^3 component ablation (test split, single seed). Components: backbone B (RepFasterNet), neck C (HS-FPN), fusion D (`DRBC3`). Best value per column in bold.

Configuration	Components	mAP@0.5	mAP@[.5:.95]	Params	FPS
A (baseline r18)	none	0.763	0.514	19.9 M	60.7
A+B	B	0.864	0.577	16.8 M	59.6
A+C	C	0.712	0.421	18.7 M	54.6
A+D	D	0.838	0.566	18.2 M	62.9
A+B+C	B,C	0.666	0.411	15.6 M	56.1
A+B+D	B,D	0.790	0.541	15.1 M	64.6
A+C+D	C,D	0.854	0.587	17.4 M	59.0
A+B+C+D (full)	B,C,D	0.762	0.489	**14.3 M**	56.5

parameter count (14.3 M), matching the efficiency (not accuracy) finding of the initialisation-controlled comparison. All values are single seed; a multi-seed run would add error bars and is the one remaining optional extension.

The component story does not transfer cleanly from the diamond domain. The source paper's own Table 2 (measured on its binary synthetic-diamond data, mAP@0.5) tells a different story from ours, and the contrast is the heart of the cross-domain answer to RQ2. On diamonds, the neck (C) is the paper's strongest single component, lifting recall by 5.4 points and giving the largest mAP gain, and the full model A+B+C+D is the best configuration (90.6). On magnetic tiles, C is the *worst* thing to add, and the full model is no longer best. Table 9 places the two side by side.

Table 9. Component transfer, diamond domain (paper Table 2, mAP@0.5) versus magnetic tiles (ours). The ranking of configurations differs: on diamonds the full model wins and the neck (C) helps; on tiles the neck hurts and a smaller subset wins. Best per column in bold.

		Diamond (paper)		Tiles (ours)	
Config	Comp.	mAP@0.5	R	mAP@0.5	mAP@[.5:.95]
A	none	88.0	87.9	76.3	51.4
A+B	B	88.6	88.5	86.4	57.7
A+C	C	90.2	93.3	71.2	42.1
A+D	D	87.4	85.0	83.8	56.6
A+B+C	B,C	90.3	89.2	66.6	41.1
A+B+D	B,D	88.3	88.8	79.0	54.1
A+C+D	C,D	89.1	86.8	85.4	58.7
A+B+C+D (full)	B,C,D	90.6	88.3	76.2	48.9

This is the most concrete cross-domain result of the study: a component (HS-FPN) that is the source paper's largest contributor on synthetic diamonds is actively harmful on magnetic-tile defects, so the published full model does not reproduce as the best configuration here. We read this as the neck's high-level semantic screening suiting the diamond domain (few, scale-varying particles on clean backgrounds) but discarding the fine spatial detail that small, elongated tile defects depend on. The backbone (B) and fusion (D) transfer their benefit; the neck (C) does not.

The two combinations A+B+D and A+C+D were first trained during an earlier exploration

of the design and re-evaluated here; all eight configurations correspond exactly to the paper's Table 2.

7 Discussion

7.1 Transformers against CNNs

RT-DETR reaches a test mAP@0.5 of 0.763 against YOLOv8n's 0.403, a factor of 1.9. Three observations matter more than that headline.

Recall, not precision, is where the transformer wins. Precision is broadly comparable (RT-DETR 0.73 against YOLOv8n 0.61), but recall jumps from 0.35 (YOLOv8n) and 0.23 (YOLO11n) to 0.82. The CNNs are conservative: when they fire they are often right, but they miss roughly two thirds of defects. For inspection this is the costly failure mode. The set-prediction head (one query per object, Hungarian matching, no NMS) recovers defects the anchor-based heads drop.

Only the transformers learn the rare classes. Both CNNs essentially solve *blowhole* (large, round, high-contrast) and then fall off a cliff on the elongated, scarce classes. Both transformers keep every class at or above 0.36 (Table 6). Two causes plausibly compound: the AIFI attention encoder supplies global context that helps localise thin, low-contrast cracks and frays that a limited CNN receptive field struggles with, and pretraining supplies a feature prior that 32 fray images cannot teach from scratch.

Cost. The accuracy comes at roughly five times the latency (40 against 205 FPS) and seven times the parameters (19.9 against 3.0 M). At 40 FPS the transformer still clears real-time for line-scan inspection, so for a quality-critical task the trade is worth it here.

Confound, read the win carefully. RT-DETR was initialised from a diamond-domain checkpoint while the CNNs were initialised from COCO. Part of RT-DETR's advantage is therefore domain-relevant pretraining, not architecture alone. The balanced per-class behaviour is partly architectural (attention plus set matching) and partly this transfer; the two cannot be fully separated for the CNN comparison, which is precisely why we built the from-scratch transformer baseline of Section 6.1.

7.2 Diamond-DETR against RT-DETR

Read against the initialisation-controlled baseline (Section 6.1), Diamond-DETR delivers what its name promises: a lightweight detector that holds detection quality (tied mAP@0.5, marginally lower mAP@[0.5:0.95]) at 28% fewer parameters and competitive speed. This is a genuine reproduction of the paper's *efficiency* claim under cross-domain transfer; what does not reproduce on magnetic tiles is a strict accuracy improvement over a fairly-trained RT-DETR. In the four-model table Diamond-DETR is the single best model on the three hardest classes (*break* 0.502, *crack* 0.536, *blowhole* 0.670), but that table credits it with RT-DETR's pretraining handicap; the matched-init comparison is the fair one. The complete factorial ablation identifies which swaps drive the efficiency: `DRBC3` (D) helps and HS-FPN (C) hurts on this domain, while the backbone (B) helps only in isolation, so no published configuration is optimal here and the best we found (A+C+D 0.587, A+B 0.577) beats the full model (0.489). The project's overriding methodological lesson remains optimiser sensitivity: in our from-scratch, five-class setup an initial SGD run left the transformers near zero and would have wrongly concluded that "CNNs beat transformers". (The source paper reached 88 to 90% with SGD on its binary diamond task, so this is a regime-specific pitfall, not a universal rule.)

8 Grad-CAM attention analysis

To corroborate the quantitative gap, Grad-CAM [9] attention maps were computed for each trained detector on one tile per defect class, using the official `pytorch-grad-cam` library on the deepest backbone convolution. Figure 5 shows the comparison grid.

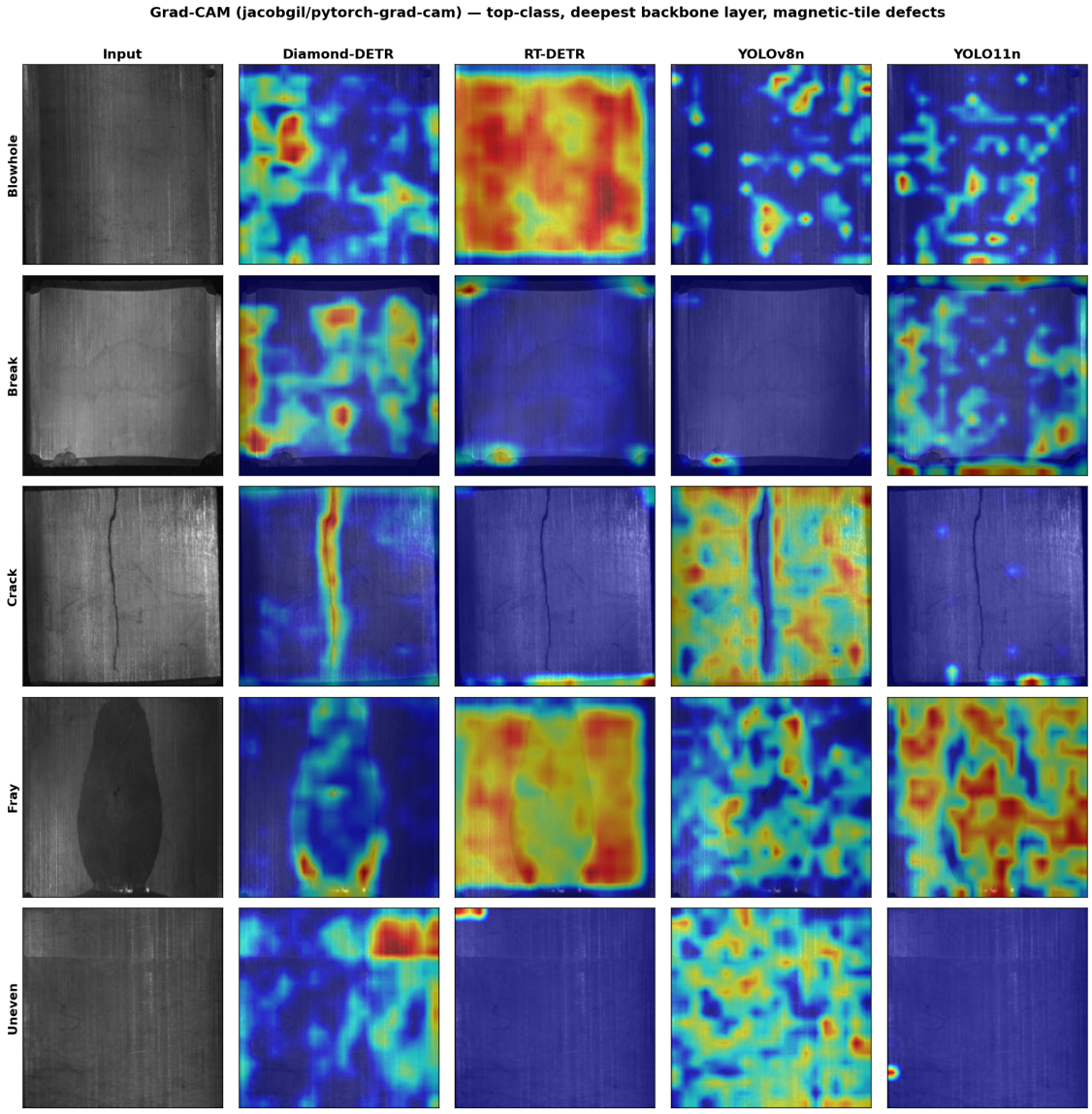


Figure 5. Grad-CAM attention overlays. Rows are the five defect classes; columns are the input and the four models. The transformers, especially Diamond-DETR, produce focused, defect-aligned maps (a sharp line along the *crack*), while the CNN maps are diffuse. The top predicted class also separates them: both transformers classify all five defects correctly, whereas YOLO11n collapses to class 0 on almost everything, mirroring its near-zero rare-class AP.

9 Cross-domain comparison

Table 10 places our magnetic-tile results beside the values the source paper reports for its synthetic-diamond domain (Tables 2 and 3 of [1], verified against the published article). The drop from the diamond figures to the tile figures is expected, since the diamond task is a binary, balanced, clean-background problem of 627 images while ours is a five-class, severely imbalanced one. The comparison is restricted to the two transformers, which are the same architectures in both studies; the paper benchmarks YOLOv8s/m and YOLOv10s/m rather than the nano CNNs we use, so

there is no matching-variant diamond figure for YOLOv8n or YOLO11n, and we do not invent one. The relative story is preserved across domains: the two transformers stay close to one another, and Diamond-DETR stays competitive with RT-DETR while being lighter. The level, however, does not transfer (Diamond-DETR 90.6 → 76.2 mAP@0.5), and crucially *which* components help changes (Table 9).

Table 10. Cross-domain comparison of the two transformers. Diamond figures are from the source paper [1] (RT-DETR is its baseline "A", Table 2/3); magnetic-tile figures are this study's test-split metrics from `results/metrics/*.json`. Values are percentages; the paper does not report mAP@[.5:.95] for the diamond domain.

Model	Domain	P	R	mAP@0.5	mAP@[.5:.95]
Diamond-DETR	Diamond (paper)	92.8	88.3	90.6	n/a
	Magnetic Tile (ours)	74.5	74.6	76.2	48.9
RT-DETR	Diamond (paper, baseline A)	89.7	87.9	88.0	n/a
	Magnetic Tile (ours)	73.3	81.9	76.3	44.6

10 Limitations

1. **Initialisation.** The headline four-model table is not initialisation-controlled (Diamond-DETR random, RT-DETR diamond-pretrained); Section 6.1 neutralises this with a from-scratch RT-DETR baseline. A fully clean test would pretrain both transformers identically.
2. **Augmentation asymmetry.** The CNNs additionally use mosaic, RandAugment, and random erasing, which the DETR fork does not. The transformers win despite seeing less augmentation, so the family result is conservative; a strict protocol would match augmentation across families.
3. **Different optimisers and epoch counts per family** (AdamW/200 against SGD/150). These are justified by model-family practice but are not controlled variables.
4. **Framework confound.** Transformers run on a forked Ultralytics 8.0.201, CNNs on stock 8.4.x; hyper-parameters were matched manually.
5. **Mask to box is lossy.** Touching defect regions can merge into a single box.
6. **No explicit class rebalancing**, a deliberate choice to keep the comparison clean; loss reweighting and oversampling are strong follow-ups.
7. **Single seed** (42). Mean and standard deviation over several seeds would be needed to claim that the 0.762 against 0.763 difference is significant; on a 138-image test split it is almost certainly within noise.

11 Future work

The from-scratch RT-DETR-r18 baseline that makes the central comparison initialisation-controlled is now complete (Section 6.1), and the complete 2^3 component ablation now quantifies each swap's contribution (Table 8). The immediate next step is multi-seed runs with bootstrap confidence intervals so that the per-component differences can be called significant or not; beyond that are matching augmentation across families to remove that confound; a true multi-domain evaluation on NEU-DET [13] and similar datasets; class-imbalance methods targeted at the rare defects, such as focal loss [10], class-balanced re-weighting [11], and copy-paste augmentation [12]; an instance-segmentation framing that uses the original masks and reports mask IoU; and deployment work (quantisation, distillation) to map a fuller accuracy versus latency frontier.

12 Conclusion

On the Magnetic Tile defect dataset the transformer detectors clearly outperform both compact CNNs (test mAP@0.5 0.763 against 0.403 and 0.335) and, decisively for inspection, raise recall to 0.82 against 0.23 to 0.35 with balanced per-class accuracy where the CNNs learn only the easy blowhole class. On the study's central question, the initialisation-controlled comparison shows that Diamond-DETR's three component swaps reproduce the paper's *efficiency* claim cross-domain: near-identical detection quality (tied mAP@0.5, marginally lower mAP@[0.5:0.95]) at 28% fewer parameters, rather than a strict accuracy improvement over a fairly-trained RT-DETR. A hand-designed lightweight transformer thus transfers its efficiency to a new domain even when its accuracy edge does not. Two honest lessons frame the work: in this from-scratch, five-class regime the transformers needed AdamW, since an initial SGD run left both near zero and would have produced the opposite, wrong conclusion (the source paper trained with SGD on its easier binary task, so the sensitivity is regime-specific); and an experimental design must reproduce a paper's configurations exactly, as our completion of the full eight-configuration ablation, matching the paper's Table 2, illustrates.

A Data integrity and engineering notes

The study was rebuilt several times as problems were found and fixed. The episodes below are recorded both for honesty and because they explain non-obvious choices in the main text.

- **Data leakage caught and removed.** An early run reused a stale output directory, so an older binary-class split was overlaid on the five-class split, creating 624 duplicate images (385 shared between train and validation). The data-preparation script now wipes its output directory before writing; the split was regenerated and verified to have zero image overlap and instance counts matching the source. No contaminated number appears in this report.
- **GPU memory fixes the batch size.** Batch 8 caused a CUDA out-of-memory on the 8 GB card when training reached RT-DETR (the transformer peaks near 6.3 GB at batch 4). Batch 8 does not fit and batch 2 needlessly deviates from the paper, so all four models share the paper's intended batch 4, verified to fit.
- **Fork compatibility.** The vendored Ultralytics 8.0.201 does not run on modern PyTorch 2.8 and NumPy 2.4 unchanged. Six compatibility patches were applied (Appendix B); they are compatibility fixes only and change no model. Without them the DETR models silently never produced metrics, and an early "transformers fail" impression was in fact a CNN-only run.
- **Architecture correction and weight recovery.** The local `Diamond-detr.yaml` had been replaced with a non-paper FasterNet-T2 backbone variant (23.7 M parameters). The real paper architecture (`BasicBlock_Faster_Block_Rep` ResNet-18 variant, 14.3 M) was restored from a fresh clone of the source repository, which also yielded RT-DETR's diamond-pretrained checkpoint (39 MB, mAP@0.5 0.89) used as RT-DETR's init. No Diamond-DETR trained weights are published anywhere, so Diamond-DETR trains from random init.
- **The optimiser lesson.** An initial SGD run left both transformers near zero mAP. Switching to AdamW, the standard DETR-family optimiser, let RT-DETR converge from pretrained weights and Diamond-DETR converge from scratch (validation mAP@0.5 climbed from about 0.00001 to 0.80). This is the single most important methodological lesson of the project: the CNN defaults of SGD and COCO initialisation do not transfer to transformer detectors.
- **Ablation design: mis-scoped, then matched to the paper.** We first explored a leave-one-out subset, then briefly narrowed the reproduction to six "additive" rows under the mistaken belief that the paper's Table 2 was additive. On re-reading the source paper, its Table 2 in fact reports the complete eight-configuration 2^3 factorial (including A+B+D and A+C+D).

Our final ablation reinstates all eight and matches the paper's design exactly; no configuration is invented and none is dropped.

B Fork compatibility patches

The following six patches were applied in place to the vendored fork at `code/diamond_detr/RTDETR-main`. They restore compatibility with PyTorch 2.8 and NumPy 2.4 and fix one latent bug; none alter a model definition.

#	File	Fix
1	`utils/metrics.py`	`np.trapz` to `np.trapezoid` (NumPy 2.0 removed `trapz`); restores average-precision computation
2	`utils/torch_utils.py`	`torch.load(..., weights_only=False)` (PyTorch 2.6 changed the default to `True`)
3	`nn/tasks.py`	the same `weights_only=False` at two more call sites
4	`nn/tasks.py`	`attempt_load_one_weight` returns `(model, ckpt)`, not just `model` (latent fork bug)
5	`cfg/.../Diamond-detr.yaml`	restored the real paper architecture; keep the top-level `nc` key (its absence raises `KeyError`)
6	`nn/tasks.py`	`ConvNormLayer` fuse guard (`hasattr(m, 'norm')`) so a repeated `fuse()` from the FPS benchmark plus `val()` is a safe no-op

References

[1] X. Yan, S. Yang, S. Zhang, X. Li, A. Li. "Diamond-DETR: lightweight real-time quality evaluation algorithm for synthetic diamonds." *Scientific Reports* **16**, 10711 (2026). DOI: 10.1038/s41598-026-44951-1. Source paper: the architecture under test, originally trained on a synthetic-diamond quality dataset and reproduced here cross-domain on magnetic-tile defects.

[2] Y. Zhao et al. "DETRs Beat YOLOs on Real-time Object Detection." *CVPR*, 2024. arXiv:2304.08069. DOI: 10.1109/CVPR52733.2024.01605.

[3] N. Carion et al. "End-to-End Object Detection with Transformers." *ECCV*, 2020. DOI: 10.1007/978-3-030-58452-8_13.

[4] J. Chen et al. "Run, Don't Walk: Chasing Higher FLOPS for Faster Neural Networks." *CVPR*, 2023. DOI: 10.1109/CVPR52729.2023.01157. (FasterNet / PConv; the basis of Diamond-DETR's backbone block B.)

[5] X. Ding et al. "RepVGG: Making VGG-style ConvNets Great Again." *CVPR*, 2021. DOI: 10.1109/CVPR46437.2021.01352. (Structural re-parameterisation used in blocks B and D.)

[6] Y. Chen et al. "Accurate leukocyte detection based on Deformable-DETR and multi-level feature fusion for aiding diagnosis of blood diseases." *Computers in Biology and Medicine* **170**, 107917 (2024). DOI: 10.1016/j.compbiomed.2024.107917. (Origin of the HS-FPN encoder, Diamond-DETR's neck component C; cited as ref. 19 in the source paper.)

[7] Y. Huang, C. Qiu, K. Yuan. "Surface defect saliency of magnetic tile." *The Visual Computer* **36**, 85-96 (2020). DOI: 10.1007/s00371-018-1588-5.

[8] G. Jocher et al. *Ultralytics YOLO* (YOLOv8, YOLO11). https://github.com/ultralytics/ultralytics.

[9] R. R. Selvaraju et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization." *ICCV*, 2017.

[10] T.-Y. Lin et al. "Focal Loss for Dense Object Detection." *ICCV*, 2017.

[11] Y. Cui et al. "Class-Balanced Loss Based on Effective Number of Samples." *CVPR*, 2019.

[12] G. Ghiasi et al. "Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation." *CVPR*, 2021.

[13] K. Song, Y. Yan. "A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects (NEU surface defect database)." *Applied Surface Science*, 2013.